

AI 聊天功能实现详解

AI 聊天功能实现详解

一、调用的是哪家公司的 AI 接口？

项目未直接调用 OpenAI 等大公司的公开接口，而是通过 “自定义 API 服务” 中转，具体说明如下：

- **接口基础地址**：代码中定义的 API 基础 URL 为自定义服务地址：

```
private let baseURL = "http://api.test.connectthedots.life" // 替换为实际的API基础URL
```

（文件：AgentService.swift）

- **模型支持**：明确支持 gpt-4.1 模型，说明后端服务很可能对接了 OpenAI 的 GPT 模型，前端通过自有服务器中转调用：

```
// 从 UserDefaults 获取选择的模型
let selectedModel = UserDefaults.standard.string(forKey: "selectedModel") ?? "gpt-4.1"
```

（文件：AgentService.swift）

二、数据获取的完整流程

AI 聊天的数据交互通过 AgentService 类实现，该类作为 “前端与 AI 服务器的中间人”，处理消息发送与接收，核心流程如下：

1. 发送消息的两种方式

支持 “文本消息” 和 “语音消息” 两种交互形式，分别对应不同的 API 接口，但最终均通过流式连接接收回复：

（1）文本消息发送

调用 `chatWithAgent` 方法，向 `/api/agent/chat` 接口发送文本内容：

```
func chatWithAgent(message: String, messageId: UUID) async throws ->
AsyncThrowingStream<StreamMessage, Error> {
    guard let url = URL(string: "\(baseUrl)/api/agent/chat") else {
        throw NetworkError.invalidURL
    }
    // 构建请求（包含文本内容、模型选择等参数）...
    return try await streamRequest(with: request)
}
```

（文件：AgentService.swift）

（2）语音消息发送

调用 `sendVoice` 方法，向 `/api/agent/voice` 接口发送音频数据（`multipart/form-data` 格式）：

```
func sendVoice(audioData: Data, messageId: UUID) async throws ->
AsyncThrowingStream<StreamMessage, Error> {
    guard let url = URL(string: "\(baseUrl)/api/agent/voice") else {
        throw NetworkError.invalidURL
    }
    // 构建multipart/form-data请求（包含音频数据、消息ID等）...
    return try await streamRequest(with: request)
}
```

（文件：AgentService.swift）

2. 流式接收 AI 回复（核心特性）

采用“流式传输”技术，AI 回复会以“片段”形式实时返回，避免用户等待完整回复，实现“边生成边显示”的效果，核心逻辑如下：

（1）建立流式连接

通过 `streamRequest` 方法创建 `URLSessionDataTask`，建立与服务器的长连接，并注册后台任务（确保应用在后台时仍能接收数据）：

```

private func streamRequest(with request: URLRequest) async throws ->
AsyncThrowingStream<StreamMessage, Error> {
    // 注册后台任务，确保应用在后台也能继续接收数据
    registerBackgroundTask()

    return AsyncThrowingStream { continuation in
        let task = self.session.dataTask(with: request)
        // 存储continuation以便在delegate方法中使用
        self.activeStreamContinuations[task] = continuation
        // ...
        task.resume()
    }
}

```

(文件: `AgentService.swift`)

(2) 处理流式数据

通过 `URLSessionDataDelegate` 监听数据接收事件，每次收到部分数据后，调用 `processSSEData` 方法解析（基于 SSE 协议，即 Server-Sent Events），并通过 `continuation` 实时返回消息片段：

```

// 收到部分数据时处理
func urlSession(_ session: URLSession, dataTask: URLSessionDataTask, didReceive data: Data) {
    print("收到部分数据: \(data.count) 字节")
    processSSEData(data, for: dataTask)
}
// 解析流式数据并返回消息片段
private func processSSEData(_ data: Data, for task: URLSessionDataTask) {
    // 1. 解析数据为StreamMessage对象（包含AI回复的片段内容）
    // 2. 通过之前保存的continuation，将消息片段发送给调用方
    if let continuation = activeStreamContinuations[task] {
        continuation.yield(parsedMessage) // 实时返回片段
    }
}

```

(文件: `AgentService.swift`)

三、数据如何反馈到界面

AI 回复数据从服务器到界面显示，需经过 “数据模型定义→控制器中转→本地存储” 三个环节，确保数据流转清晰：

1. 数据模型定义（Message 结构体）

定义 Message 结构体统一表示聊天消息，包含消息内容、类型、发送方等核心属性：

```
public struct Message: Identifiable, Equatable, Codable {
    enum MessageType: String, Codable {
        case text          // 纯文本消息
        case taskSuggestion // 任务建议
        case taskCreated    // 任务创建结果
        case taskCompleted  // 任务完成反馈
        case error          // 错误提示
        case thinking       // AI思考中
        case thought        // AI思考内容
        case action         // 操作指令
        case response       // 正式回复
    }

    public let id: UUID      // 消息唯一标识
    let content: String      // 消息内容
    let isUser: Bool        // 是否为用户发送（true=用户，false=AI）
    let timestamp: Date      // 发送时间
    let isThinking: Bool     // AI是否正在思考
    let messageType: MessageType // 消息类型
    // ...其他属性（如语音时长、关联任务ID等）
}
```

（文件：ChatInputView.swift）

2. 控制器作为中间人（ChatInputController）

ChatInputController 是连接 “服务层（AgentService）” 与 “界面层（ChatInputView）” 的核心，负责管理消息状态并同步更新：

- **初始化：**在 ChatInputView 中创建，接收初始消息列表、消息服务等参数：

```
self._chatController = StateObject(wrappedValue: ChatInputController(  
    messages: messages.wrappedValue,  
    messageService: messageService,  
    onRefresh: onRefresh  
))
```

(文件: ChatInputView.swift)

◦ **核心作用:**

- 调用 `AgentService` 发送消息，接收流式回复；
- 将回复片段组装为 `Message` 对象，更新内部 `messages` 数组；
- 通过 `@Published` 修饰 `messages`，确保变化时通知界面。

3. 本地存储聊天记录 (MessageService)

`MessageService` 负责将聊天记录持久化到本地（如 `UserDefaults`），支持加载历史消息，核心逻辑如下：

```
func saveMessages(_ messages: [Message]) {  
    // 只保存最近24小时的消息（优化存储，避免数据过大）  
    let recentMessages = messages.filter {  
        $0.timestamp >= Calendar.current.date(byAdding: .day, value: -1, to: Date())!  
    }  
  
    let encoder = JSONEncoder()  
    if let encoded = try? encoder.encode(recentMessages) {  
        // 使用基于用户ID的键保存消息，确保多用户数据隔离  
        userDefaults.set(encoded, forKey: messageKeyForCurrentUser())  
    }  
}  
  
// 生成用户专属的存储键（如"user_123_chat_messages"）  
private func messageKeyForCurrentUser() -> String {  
    guard let userId = UserDefaults.standard.string(forKey: "currentUserId") else {  
        return "default_chat_messages"  
    }  
    return "\\(userId)_chat_messages"  
}
```

(文件: `MessageService.swift`)

四、界面刷新的实现

界面实时刷新依赖 “流式更新 + 状态绑定 + 应用状态处理”，确保 AI 回复实时显示且体验连贯：

1. 流式更新（核心机制）

`AgentService` 接收的 AI 回复片段，通过 `continuation.yield()` 实时传递给 `ChatInputController`，控制器更新 `messages` 数组，触发界面刷新：

```
// 在processSSEData方法中，实时返回消息片段
continuation.yield(message) // 把消息片段发送给ChatInputController
```

(文件: `AgentService.swift`)

2. 状态绑定更新（SwiftUI 特性）

`ChatInputView` 通过 `onChange` 监听 `chatController.messages` 的变化，当消息列表更新时，同步更新界面的 `messages` 属性，SwiftUI 自动重新渲染：

```
// 在ChatInputView中监听控制器消息变化
.onChange(of: chatController.messages) { _, newMessages in
    print(" Controller messages changed in ChatInputView - \(newMessages.count) messages")
    // 同步父视图的绑定数据与控制器消息
    messages = newMessages
}
```

(文件: `ChatInputView.swift`)

3. 应用状态变化时的处理

针对 “应用进入后台、返回前台” 等场景，确保聊天过程不中断，核心逻辑如下：

```
// 当应用进入后台时
@objc private func handleAppDidEnterBackground() {
```

```
print(" 应用进入后台，请求后台执行时间")
registerBackgroundTask() // 延长后台执行时间，确保数据接收完成
}
// 当应用即将返回前台时
@objc private func handleAppWillEnterForeground() {
    print(" 应用返回前台")
    // 如果有任务在等待重新连接，尝试重新连接
    attemptReconnections()
    // ...其他恢复逻辑（如刷新消息状态）
}
```

（文件：AgentService.swift）

五、完整流程总结

以“用户发送文本消息并接收 AI 回复”为例，完整流程如下：

1. **用户输入**：在 ChatInputView 的输入框输入“今天天气怎么样？”，点击发送按钮；
2. **发送请求**：ChatInputController 调用 AgentService.chatWithAgent() 方法，携带文本内容发送请求；
3. **建立连接**：AgentService 创建 URLSessionDataTask，与服务器建立流式连接，注册后台任务；
4. **接收回复**：服务器返回 AI 回复片段，urlSession(_:dataTask:didReceive:) 方法触发，-processSSEData 解析片段；
5. **更新界面**：解析后的消息片段通过 continuation.yield() 传递给 ChatInputController，控制器更新 messages 数组；
6. **状态同步**：ChatInputView 的 onChange 监听 messages 变化，同步更新界面，用户看到 AI 回复实时显示（打字机效果）；
7. **本地保存**：对话结束后，MessageService 将聊天记录（最近 24 小时）保存到本地，供下次打开时加载。

通过以上流程，实现了“实时交互、状态同步、数据持久化”的完整 AI 聊天功能。

（注：文档部分内容可能由 AI 生成）